Sola Optimization Documentation

Optimization Problem Formulation

Sola provides MATLAB classes for solving constrained optimization problems of the form

$$\begin{aligned} &\min_{\boldsymbol{u},\boldsymbol{z}} J(\boldsymbol{u},\boldsymbol{z}) \qquad \Longleftrightarrow \qquad \min_{\boldsymbol{z}} \ \hat{J}(\boldsymbol{z}) = J(\mathcal{S}(\boldsymbol{z}),\boldsymbol{z}), \\ &\text{s.t. } \boldsymbol{c}(\boldsymbol{u},\boldsymbol{z}) = \boldsymbol{0}, \end{aligned} \tag{1}$$

where

$$\boldsymbol{u} \in \mathbb{R}^{n_u}, \qquad \boldsymbol{z} \in \mathbb{R}^{n_z},$$

$\boldsymbol{u}$ is the state variable, $\boldsymbol{z}$ is the control variable, $J(\boldsymbol{u},\boldsymbol{z})$ is the objective, $\boldsymbol{c}(\boldsymbol{u},\boldsymbol{z})$ defines the constraints, and $\mathcal{S}(\boldsymbol{z})$ is the state solution operator satisfying

$$\boldsymbol{c}(\mathcal{S}(\boldsymbol{z}),\boldsymbol{z}) = \boldsymbol{0}.$$

Its primary purpose is to prototype optimization constrained by differential equations, where (1) is written in a discretized form, e.g., after spatial discretization of a partial differential equation.

Within `sola/src/optimization`, the main MATLAB classes are:

- `Objective`: represents $J(\boldsymbol{u},\boldsymbol{z})$,
- `Constraint`: represents $\boldsymbol{c}(\boldsymbol{u},\boldsymbol{z})$ and the associated solution operator $\mathcal{S}$,
- `Reduced_Space_Optimization`: represents and solves the reduced problem

$$\min_{\boldsymbol{z}} J(\mathcal{S}(\boldsymbol{z}),\boldsymbol{z}).$$

The classes `Objective` and `Constraint` have interfaces to implement the functions and their first and second derivatives. The class `Reduced_Space_Optimization` takes inputs of type `Objective` and `Constraint`, and automates the implementation of adjoint equations and interfacing with MATLAB's optimization algorithms.

Sola also provides specialized classes for a broad class of optimization problems in which the state depends on time. The interface for such time-dependent problems involves the classes:

- `Dynamic_Objective`,
- `Dynamic_Constraint`.

`Dynamic_Objective` is a specialized objective class for problems of the form

$$J(\boldsymbol{u},\boldsymbol{z}) = \int_0^T g(\boldsymbol{y}(t),t)\,\mathrm{d}t + R(\boldsymbol{z}),$$

where the time-dependent state is denoted by $\boldsymbol{y}(t)$.

`Dynamic_Constraint` is a specialized constraint class for systems governed by ordinary differential equations of the form

$$\frac{\mathrm{d}}{\mathrm{d}t}\boldsymbol{y}(t) = \boldsymbol{f}(\boldsymbol{y}(t), \boldsymbol{z}, t)$$
$$\boldsymbol{y}(0) = h(\boldsymbol{z}).$$

Implementation Notes

Abstract classes

An abstract class defines methods that must be implemented before the class can be instantiated. In Sola, abstract classes provide a uniform interface for optimization algorithms while allowing users to supply problem-specific physics and derivatives. The code naming convention is that a name such as `J_uz_Apply` means that the method applies the (u, z) second derivative matrix of $J(u, z)$ to a vector. The input and output names such as `z_in` and `u_out` indicate the spaces (state or optimization variable) of the input and output vectors in the matrix-vector product.

Vectorized apply methods

Many derivative application methods must accept multiple test directions at once, arranged columnwise in a matrix. This vectorized behavior is required to interface properly with MATLAB optimization routines.

Gauss–Newton option

When the property

```
Gauss_Newton_Hess = true
```

is enabled in `Reduced_Space_Optimization`, the optimization uses a Gauss–Newton approximation of the reduced Hessian. This can simplify implementation because certain second-derivative constraint operators are no longer required.

Automatic Differentiation

The code in `sola/src/automatic_differentiation` provides an interface to automate the implementation of derivatives. Specifically, it requires that ADiGator has been downloaded from `https://github.com/matt-weinstein/adigator` and installed. The classes and functions in `sola/src/automatic_differentiation` require user implementation of the objective function and constraint, and utilize ADiGator to implement the derivatives. This is an optional module intended to facilitate rapid prototyping. However, manual implementation of the derivatives will typically give better performance.

Optimization Under Uncertainty

The code in `sola/src/optimization_under_uncertainty` provides an interface to solve optimization problems of the form

$$\min_{\boldsymbol{z}} \hat{J}(\boldsymbol{z}) = \mathbb{E}_{\boldsymbol{\theta}} \left[J(\mathcal{S}(\boldsymbol{z}, \boldsymbol{\theta}), \boldsymbol{z}) \right], \tag{2}$$

where $\boldsymbol{\theta}$ is a vector of uncertain parameters that enter the constraint, e.g., $\boldsymbol{\theta}$ corresponds to uncertainty parameters in the partial differential equation that $\mathcal{S}$ solves. In Sola, this is defined through the class `Parametric_Constraint`, which generalizes the `Constraint` base class by including $\boldsymbol{\theta}$ dependence. A cell of `Parametric_Constraint` objects, each corresponding to a different $\boldsymbol{\theta}$ sample, is passed into the class `Reduced_Space_Optimization_Under_Uncertainty`, which implements adjoints and interfaces with MATLAB's optimization algorithms to solve problems of the form (2) via a sample average approximation of the expected value in the objective function.